

MultiStepLR#

https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.MultiStepLR.html

Description:

Decays the learning rate of each parameter group by gamma once the number of epoch reaches one of the milestones. Notice that such decay can happen simultaneously with other changes to the learning rate from outside this scheduler. When last_epoch=-1, sets initial lr as lr. optimizer (Optimizer) – Wrapped optimizer. milestones (list) – List of epoch indices. Must be increasing. gamma (float) – Multiplicative factor of learning rate decay. Default: 0.1.

Function Signature

```
class torch.optim.lr_scheduler.MultiStepLR(optimizer,  
milestones, gamma=0.1, last_epoch=-1)[source]#
```

Parameters

```
get_last_lr()[source]#
```

Return type

```
get_lr()[source]#
```

Returns:

dict[str, Any]

Code Examples

```
>>> # Assuming optimizer uses lr = 0.05 for all groups
>>> # lr = 0.05      if epoch < 30
>>> # lr = 0.005     if 30 <= epoch < 80
>>> # lr = 0.0005    if epoch >= 80
>>> scheduler = MultiStepLR(optimizer, milestones=[30, 80],
gamma=0.1)
>>> for epoch in range(100):
>>>     train(...)
>>>     validate(...)
>>>     scheduler.step()
```

Detailed Documentation

Documentation

Decays the learning rate of each parameter group by gamma once the number of epoch reaches one of the milestones.

Notice that such decay can happen simultaneously with other changes to the learning rate from outside this scheduler. When `last_epoch=-1`, sets initial lr as lr.

`optimizer (Optimizer)` – Wrapped optimizer.

`milestones (list)` – List of epoch indices. Must be increasing.

`gamma (float)` – Multiplicative factor of learning rate decay. Default: 0.1.

`last_epoch (int)` – The index of last epoch. Default: -1.

Return last computed learning rate by current scheduler.

Compute the learning rate of each parameter group.

Load the scheduler's state.

`state_dict (dict)` – scheduler state. Should be an object returned from a call to `state_dict()`.

Return the state of the scheduler as a dict.

It contains an entry for every variable in `self.__dict__` which is not the optimizer.

